Course Title: Cryptography and Network Security Lab	**Course Code**: BCSE309P
Faculty: Prof. Karthika V	Slot: L15+L16
Regno: *23BCE1344*	Name: *Aditya*

EX6

Man in the Middle Attack in Diffie Hellman Key Exchange

File name: Ex6 _23BCE1344.pdf

Codes

Cypto_functions.py:

```
import random

def is_prime(n, k=40):
    if n < 2:
        return False
    if n == 2 or n == 3:
        return True
    if n % 2 == 0:
        return False
    r, d = 0, n - 1
    while d % 2 == 0:
        r += 1
        d //= 2
    for _ in range(k):
        a = random.randint(2, n - 2)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True

def generate_prime(bits=512):
    while True:
        q = random.getrandbits(bits)
        q |= (1 << bits - 1) | 1
        if is_prime(q):
            return q

def generate_private_key(q):
    return random.randint(2, q - 2)

def compute_public_key(alpha, private_key, q):
    return pow(alpha, private_key, q)
```

```python
def compute_shared_key(public_key, private_key, q):
    return pow(public_key, private_key, q)

def encrypt_message(message, key):
    encrypted = ""
    key_str = str(key)
    for i, char in enumerate(message):
        shift = int(key_str[i % len(key_str)])
        encrypted += chr(ord(char) + shift)
    return encrypted

def decrypt_message(encrypted, key):
    decrypted = ""
    key_str = str(key)
    for i, char in enumerate(encrypted):
        shift = int(key_str[i % len(key_str)])
        decrypted += chr(ord(char) - shift)
    return decrypted
```

Alice:

```
import socket
import threading
import crypto_functions as cf

def receive_messages(client, key, running):
    while running[0]:
        try:
            encrypted = client.recv(4096).decode().strip()
            if not encrypted:
                break
            decrypted = cf.decrypt_message(encrypted, key)
            print(f"\nBob: {decrypted}")
            print("You: ", end="", flush=True)
        except:
            break

def main():
    print("=" * 50)
    print("ALICE")
    print("=" * 50)

    q = int(input("Enter prime number (q): "))
    alpha = int(input("Enter primitive root (alpha): "))

    Xa = cf.generate_private_key(q)
    print(f"Private key (Xa): {Xa}")

    Ya = cf.compute_public_key(alpha, Xa, q)
    print(f"Public key (Ya): {Ya}")

    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect(("localhost", 8001))

    client.sendall(str(Ya).encode() + b'\n')

    data = client.recv(4096).decode().strip()
    Yd = int(data)
    print(f"Received public key from Bob: {Yd}")

    Ka = cf.compute_shared_key(Yd, Xa, q)
    print(f"Shared key: {Ka}")
```

```
    print("\n" + "=" * 50)
    print("CHAT STARTED (type 'exit' to quit)")
    print("=" * 50 + "\n")

    running = [True]
    receiver = threading.Thread(target=receive_messages, args=(client, Ka,
running))
    receiver.daemon = True
    receiver.start()

    try:
        while True:
            message = input("You: ")
            if message.lower() == 'exit':
                running[0] = False
                break
            encrypted = cf.encrypt_message(message, Ka)
            client.sendall(encrypted.encode() + b'\n')
    except KeyboardInterrupt:
        running[0] = False

    client.close()

if __name__ == "__main__":
    main()
```

Bob:

```
import socket
import threading
import crypto_functions as cf

def receive_messages(client, key, running):
    while running[0]:
        try:
            encrypted = client.recv(4096).decode().strip()
            if not encrypted:
                break
            decrypted = cf.decrypt_message(encrypted, key)
            print(f"\nAlice: {decrypted}")
            print("You: ", end="", flush=True)
        except:
            break

def main():
    print("=" * 50)
    print("BOB")
    print("=" * 50)

    q = int(input("Enter prime number (q): "))
    alpha = int(input("Enter primitive root (alpha): "))

    Xb = cf.generate_private_key(q)
    print(f"Private key (Xb): {Xb}")

    Yb = cf.compute_public_key(alpha, Xb, q)
    print(f"Public key (Yb): {Yb}")

    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect(("localhost", 8002))

    client.sendall(str(Yb).encode() + b'\n')

    data = client.recv(4096).decode().strip()
    Yd = int(data)
    print(f"Received public key from Alice: {Yd}")

    Kb = cf.compute_shared_key(Yd, Xb, q)
    print(f"Shared key: {Kb}")
```

```
    print("\n" + "=" * 50)
    print("CHAT STARTED (type 'exit' to quit)")
    print("=" * 50 + "\n")

    running = [True]
    receiver = threading.Thread(target=receive_messages, args=(client, Kb,
running))
    receiver.daemon = True
    receiver.start()

    try:
        while True:
            message = input("You: ")
            if message.lower() == 'exit':
                running[0] = False
                break
            encrypted = cf.encrypt_message(message, Kb)
            client.sendall(encrypted.encode() + b'\n')
    except KeyboardInterrupt:
        running[0] = False

    client.close()

if __name__ == "__main__":
    main()
```

attacker:

```
import socket
import threading
import crypto_functions as cf

class MITMAttacker:
    def __init__(self, q, alpha):
        self.q = q
        self.alpha = alpha
        self.Xd1 = cf.generate_private_key(q)
        self.Xd2 = cf.generate_private_key(q)
        self.Ka = None
        self.Kb = None
        self.conn_alice = None
        self.conn_bob = None

    def setup_alice(self, conn):
        self.conn_alice = conn
        data = conn.recv(4096).decode().strip()
        Ya = int(data)
        print(f"\n[KEY EXCHANGE] Received Ya from Alice: {Ya}")

        Yd1 = cf.compute_public_key(self.alpha, self.Xd1, self.q)
        print(f"[KEY EXCHANGE] Sending Yd1 to Alice: {Yd1}")
        conn.sendall(str(Yd1).encode() + b'\n')

        self.Ka = cf.compute_shared_key(Ya, self.Xd1, self.q)
        print(f"[KEY EXCHANGE] Shared key with Alice (Ka): {self.Ka}")

    def setup_bob(self, conn):
        self.conn_bob = conn
        data = conn.recv(4096).decode().strip()
        Yb = int(data)
        print(f"\n[KEY EXCHANGE] Received Yb from Bob: {Yb}")

        Yd2 = cf.compute_public_key(self.alpha, self.Xd2, self.q)
        print(f"[KEY EXCHANGE] Sending Yd2 to Bob: {Yd2}")
        conn.sendall(str(Yd2).encode() + b'\n')

        self.Kb = cf.compute_shared_key(Yb, self.Xd2, self.q)
        print(f"[KEY EXCHANGE] Shared key with Bob (Kb): {self.Kb}")
```

```python
    def listen_from_alice(self):
        print("\n[LISTENER] Alice listener thread started")
        while True:
            try:
                encrypted = self.conn_alice.recv(4096).decode().strip()
                if not encrypted:
                    break

                print(f"\n[ALICE → BOB] Encrypted: {encrypted}")
                decrypted = cf.decrypt_message(encrypted, self.Ka)
                print(f"[ALICE → BOB] Decrypted: {decrypted}")

                re_encrypted = cf.encrypt_message(decrypted, self.Kb)
                print(f"[ALICE → BOB] Re-encrypted for Bob: {re_encrypted}")
                self.conn_bob.sendall(re_encrypted.encode() + b'\n')
                print(f"[ALICE → BOB] Forwarded to Bob\n")
            except:
                break

    def listen_from_bob(self):
        print("[LISTENER] Bob listener thread started\n")
        while True:
            try:
                encrypted = self.conn_bob.recv(4096).decode().strip()
                if not encrypted:
                    break

                print(f"\n[BOB → ALICE] Encrypted: {encrypted}")
                decrypted = cf.decrypt_message(encrypted, self.Kb)
                print(f"[BOB → ALICE] Decrypted: {decrypted}")

                re_encrypted = cf.encrypt_message(decrypted, self.Ka)
                print(f"[BOB → ALICE] Re-encrypted for Alice: {re_encrypted}")
                self.conn_alice.sendall(re_encrypted.encode() + b'\n')
                print(f"[BOB → ALICE] Forwarded to Alice\n")
            except:
                break

def main():
    print("=" * 60)
    print("ATTACKER (MAN IN THE MIDDLE)")
    print("=" * 60)
```

```python
q = int(input("Enter prime number (q): "))
alpha = int(input("Enter primitive root (alpha): "))

attacker = MITMAttacker(q, alpha)

print(f"\nPrivate key for Alice (Xd1): {attacker.Xd1}")
print(f"Private key for Bob (Xd2): {attacker.Xd2}")

server_alice = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_alice.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_alice.bind(("localhost", 8001))
server_alice.listen(1)

server_bob = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server_bob.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
server_bob.bind(("localhost", 8002))
server_bob.listen(1)

print("\n" + "=" * 60)
print("Waiting for Alice on port 8001...")
print("Waiting for Bob on port 8002...")
print("=" * 60)

conn_alice, _ = server_alice.accept()
print("\n✓ Alice connected!")
attacker.setup_alice(conn_alice)

conn_bob, _ = server_bob.accept()
print("\n✓ Bob connected!")
attacker.setup_bob(conn_bob)

print("\n" + "=" * 60)
print("INTERCEPTION ACTIVE - ALL MESSAGES WILL BE LOGGED")
print("=" * 60)

alice_thread = threading.Thread(target=attacker.listen_from_alice)
bob_thread = threading.Thread(target=attacker.listen_from_bob)

alice_thread.daemon = True
bob_thread.daemon = True

alice_thread.start()
```

```
    bob_thread.start()

    try:
        alice_thread.join()
        bob_thread.join()
    except KeyboardInterrupt:
        print("\n\nAttacker shutting down...")

    conn_alice.close()
    conn_bob.close()
    server_alice.close()
    server_bob.close()

if __name__ == "__main__":
    main()
```

Output:

Alice-Bob chat:

```
(lab) ak@DevLab:~/Desktop/Data/College/Semester6/Assignment/Crypto Lab/Ex6$ python alice.py
==================================================
ALICE
==================================================
Enter prime number (q): 23
Enter primitive root (alpha): 5
Private key (Xa): 13
Public key (Ya): 21
Received public key from Bob: 19
Shared key: 7

==================================================
CHAT STARTED (type 'exit' to quit)
==================================================

You: Hello Bob, How are you?
You:
Bob: Great, Is this Alice?
You: Yes, Do you think we are safe to talk here?
You:
Bob: Yeah, why not?
You: Okay then, here is my password: (Qwerty@1234) --Make sure, No on else gets it!!
You:
Bob: Yeah sure!!
You:
```

```
(lab) ak@DevLab:~/Desktop/Data/College/Semester6/Assignment/Crypto Lab/Ex6$ python bob.py
==================================================
BOB
==================================================
Enter prime number (q): 23
Enter primitive root (alpha): 5
Private key (Xb): 11
Public key (Yb): 22
Received public key from Alice: 8
Shared key: 1

==================================================
CHAT STARTED (type 'exit' to quit)
==================================================

You:
Alice: Hello Bob, How are you?
You: Great, Is this Alice?
You:
Alice: Yes, Do you think we are safe to talk here?
You: Yeah, why not?
You:
Alice: Okay then, here is my password: (Qwerty@1234) --Make sure, No on else gets it!!
You: Yeah sure!!
You: A
```

Attacker's Interception:

```
(lab) ak@DevLab:~/Desktop/Data/College/Semester6/Assignment/Crypto Lab/Ex6$ python attacker.py
============================================================
ATTACKER (MAN IN THE MIDDLE)
============================================================
Enter prime number (q): 23
Enter primitive root (alpha): 5

Private key for Alice (Xd1): 15
Private key for Bob (Xd2): 6

============================================================
Waiting for Alice on port 8001...
Waiting for Bob on port 8002...
============================================================

✓ Alice connected!

[KEY EXCHANGE] Received Ya from Alice: 21
[KEY EXCHANGE] Sending Yd1 to Alice: 19
[KEY EXCHANGE] Shared key with Alice (Ka): 7

✓ Bob connected!

[KEY EXCHANGE] Received Yb from Bob: 22
[KEY EXCHANGE] Sending Yd2 to Bob: 8
[KEY EXCHANGE] Shared key with Bob (Kb): 1

============================================================
INTERCEPTION ACTIVE - ALL MESSAGES WILL BE LOGGED
============================================================

[LISTENER] Alice listener thread started
[LISTENER] Bob listener thread started


[ALICE → BOB] Encrypted: Olssv'Ivi3'Ov~'hyl'v|F
[ALICE → BOB] Decrypted: Hello Bob, How are you?
[ALICE → BOB] Re-encrypted for Bob: Ifmmp!Cpc-!Ipx!bsf!zpv@
[ALICE → BOB] Forwarded to Bob


[BOB → ALICE] Encrypted: Hsfbu-!Jt!uijt!Bmjdf@
[BOB → ALICE] Decrypted: Great, Is this Alice?
[BOB → ALICE] Re-encrypted for Alice: Nylh{3'Pz'{opz'HspjlF
[BOB → ALICE] Forwarded to Alice


[ALICE → BOB] Encrypted: `lz3'Kv'v|'{opur'~l'hyl'zhml'{v'{hsr'olylF
[ALICE → BOB] Decrypted: Yes, Do you think we are safe to talk here?
[ALICE → BOB] Re-encrypted for Bob: Zft-!Ep!zpv!uijol!xf!bsf!tbgf!up!ubml!ifsf@
[ALICE → BOB] Forwarded to Bob


[BOB → ALICE] Encrypted: Zfbi-!xiz!opu@
[BOB → ALICE] Decrypted: Yeah, why not?
[BOB → ALICE] Re-encrypted for Alice: `lho3'~o'uv{F
[BOB → ALICE] Forwarded to Alice


[ALICE → BOB] Encrypted: Vrh'{olu3'olyl'pz't'whzz~vykA'/X~ly{G89:;0'44Thrl'z|yl3'Uv'vu'lszl'nl{z'p{((
[ALICE → BOB] Decrypted: Okay then, here is my password: (Qwerty@1234) --Make sure, No on else gets it!!
[ALICE → BOB] Re-encrypted for Bob: Plbz!uifo-!ifsf!jt!nz!qbttxpse;!)RxfsuzA2345*!..Nblf!tvsf-!Op!po!fmtf!hfut!ju""
[ALICE → BOB] Forwarded to Bob


[BOB → ALICE] Encrypted: Zfbi!tvsf""
[BOB → ALICE] Decrypted: Yeah sure!!
[BOB → ALICE] Re-encrypted for Alice: `lho'z|yl((
[BOB → ALICE] Forwarded to Alice
```